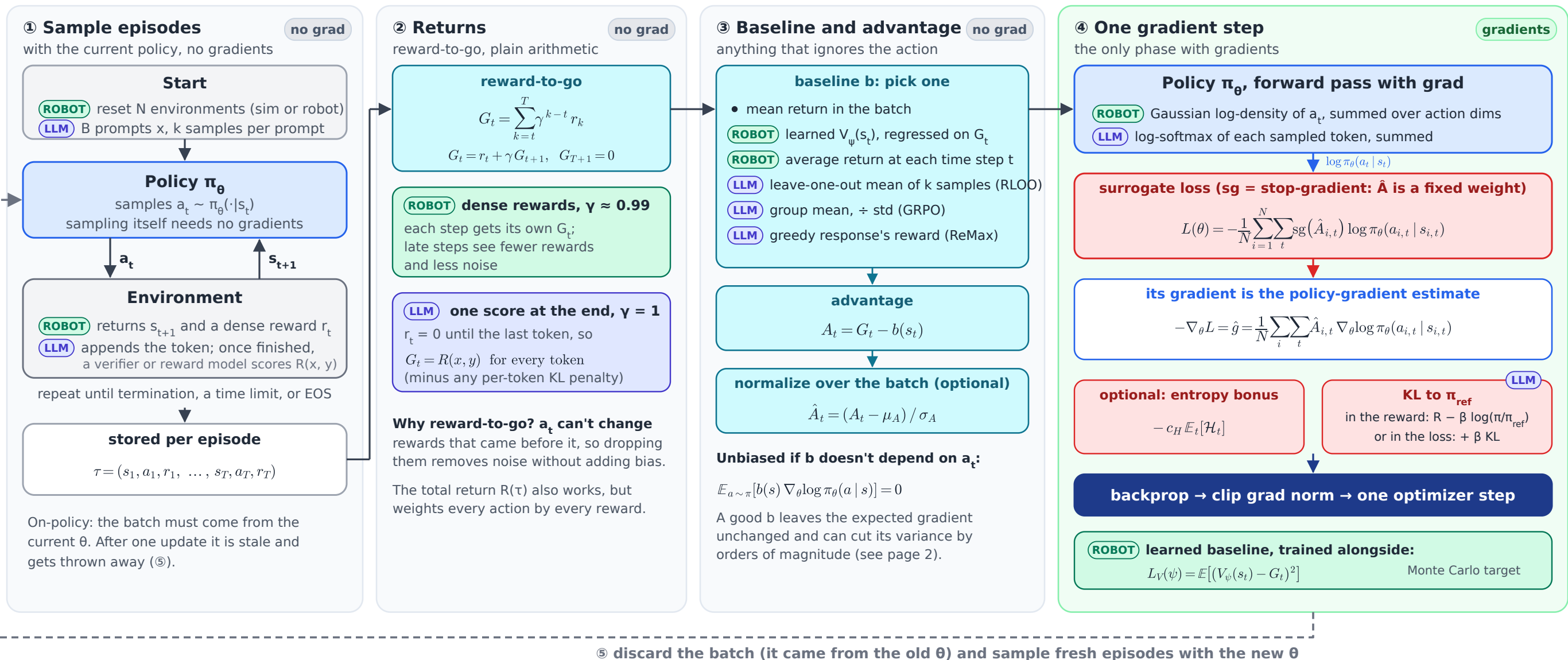


REINFORCE cheat sheet · robotics and LLM

The simplest policy-gradient method: sample episodes, score them, and raise each action's log-probability in proportion to its advantage over a baseline. Untagged parts apply to both settings.

 trainable model
 environment or scorer
 arithmetic, no grad
 loss term
 no-gradient phase
 gradient phase
 ROBOT robotics only
 LLM LLM only



Pseudocode

```
for iteration = 1, 2, ...:
```

```
sample N episodes with  $\pi_\theta$  (no grad):
```

ROBOT: N rollouts of up to T steps $\rightarrow s_t, a_t, r_t$

LLM: B prompts $\times$ k responses $\rightarrow$ tokens, $R(x, y)$

$$G_t = r_t + \gamma \cdot G_{t+1} \quad \# \text{ LLM: } G_t = R(x, y), \gamma = 1$$

b = batch mean | $V_{\psi(s_t)}$ | leave-one-out | greedy reward

$A_t = G_t - b$; optionally normalize over the batch

$$\text{loss} = -\text{mean}_i \sum_t \text{sg}(A_t) \cdot \log \pi_{\theta}(a_t | s_t)$$
$$\text{loss} += -c \, H \cdot H \quad (\text{LLM: } + \beta \cdot \text{KL to } \pi_{\text{ref}}, \text{ or KL in R})$$

```
backprop; clip grad norm; one optimizer step
```

ROBOT (learned baseline): step ψ on $(V \psi(s, t) - G_t)^2$

```
discard the batch # on-policy
```

Key facts

- Needs only log π and returns: no critic, no model of the environment (the dynamics cancel).
- Unbiased but high-variance. Baselines, reward-to-go and bigger batches are the fixes.
- On-policy: each update needs fresh samples. Reusing a batch needs importance ratios (PPO).
- A baseline is free as long as it doesn't depend on the action it is scoring.

ROBOT long horizons make credit assignment hard: a fall at step T is blamed on every earlier action.

ROBOT a learned V_{π} fit to Monte Carlo returns is still REINFORCE; bootstrapping makes it actor-critic.

LLM one score and $\gamma = 1$, so every token in a response gets the same advantage.

LLM RLOO, ReMax and GRPO (one update per batch) are REINFORCE with different baselines.

LLM “keep the correct samples, SFT on them” is REINFORCE with $A \in \{0, 1\}$ (1-0 REINFORCE).

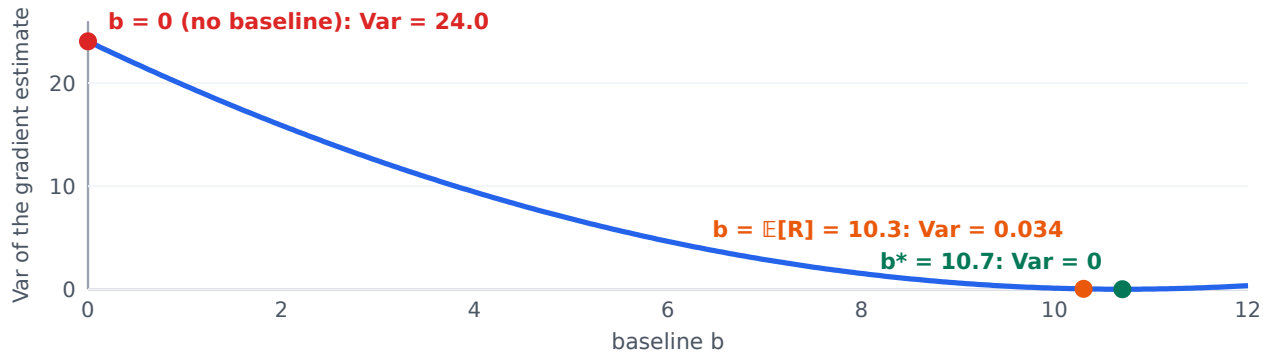
LLM if all k samples of a prompt score the same, group baselines give $A = 0$: no learning signal.

Variance, baselines, and the two settings side by side

The variance plot and the worked examples are exact for the numbers shown.

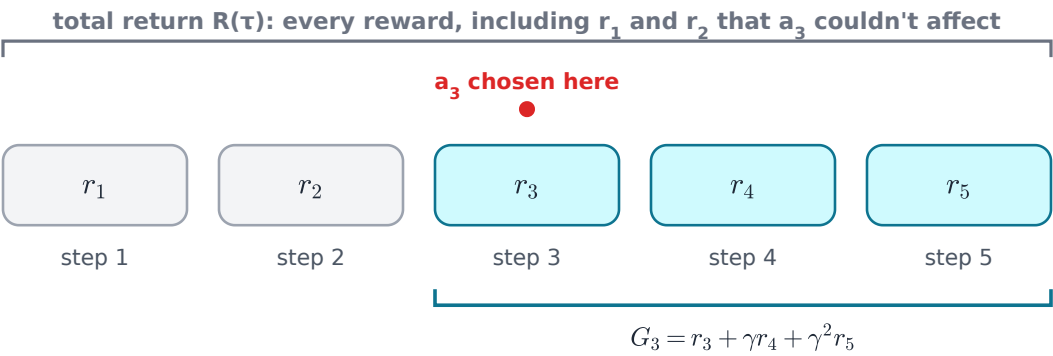
Why baselines matter (exact, one-step example with two actions)

$\pi(a = 1) = 0.3$; rewards $R(1) = 11$, $R(0) = 10$: a reward offset of 10, as with an always-positive score.



The expected gradient is 0.21 for every b; only the noise changes. The mean baseline cuts variance $\sim 700\times$.

Reward-to-go vs. total return, for the action at $t = 3$



Both give an unbiased gradient; dropping past rewards removes their noise from a_3 's weight.

LLM with a single reward at the end and $\gamma = 1$, the two are identical: $G_t = R(x, y)$.

	ROBOT Robotics / continuous control	LLM LLM fine-tuning
Episode	A T-step rollout until termination or a time limit	One response, until EOS or the maximum length
Action	Continuous vector from a Gaussian	One token from a softmax over the vocabulary
$\log \pi_{\theta}(a_t s_t)$	Gaussian log-density, summed over action dimensions	Log-softmax per token; $\log \pi(y x)$ sums over tokens
Reward	Dense, every step, from the environment	One score $R(x, y)$ at the end, from a verifier or reward model
Return G_t	Discounted reward-to-go, $\gamma \approx 0.99$	$\gamma = 1$: $G_t = R(x, y)$ for every token (minus any KL penalty)
Baseline	Learned $V_{\psi}(s_t)$, or a batch or per-time-step average	Leave-one-out (RLOO), group mean (GRPO), greedy reward (ReMax)
Samples per update	Many episodes, e.g. $N = 128$ in Ahmed et al. (2019)	Several responses per prompt (often 4 to 16) $\times$ a batch of prompts
Anchor	Usually none	KL to π_{ref} in the reward or the loss; often dropped for verifiable rewards
Main variance source	Long horizons, noisy dynamics, action noise	Sparse 0/1 rewards; groups where every sample scores the same
Critic	Optional learned baseline, never bootstrapped	None: no value model to train or keep in memory
Seen in	Ahmed et al. (2019); RL's Razor's OpenVLA runs	RLOO, ReMax, GRPO with one update per batch, 1-0 REINFORCE

ROBOT Worked example: the robot falls at the last step ($\gamma = 0.99$)

	r_t	G_t	$V_{\psi}(s_t)$	$A_t = G_t - V_{\psi}$	push on a_t
t = 1	1.0	-6.73	-5.00	-1.73	down
t = 2	1.0	-7.81	-6.50	-1.31	down
t = 3	1.0	-8.90	-8.00	-0.90	down
t = 4	-10.0	-10.00	-9.00	-1.00	down

Every action gets blamed for the fall, discounted by distance: that's the credit-assignment problem. The baseline had partly predicted the fall, so the advantages are much smaller than the raw returns. Gaussian at $t = 4$ with $\mu = 0.4$, $\sigma = 0.5$, $a_4 = 1.15$: $\partial \log \pi / \partial \mu = (a - \mu) / \sigma^2 = 3.0$, $\partial \log \pi / \partial \log \sigma = 1.25$. With $A_4 = -1.0$ the step moves μ away from 1.15 and shrinks σ : a bad, unusual action gets rarer.

LLM Worked example: $k = 4$ answers to “What is 2 + 2?”

	y_i	R_i	leave-one-out b_i	A_i (RLOO)	$R_i - \bar{R}$	ReMax
i = 1	“4”	1	0.33	+0.67	+0.5	0
i = 2	“5”	0	0.67	-0.67	-0.5	-1
i = 3	“22”	0	0.67	-0.67	-0.5	-1
i = 4	“four”	1	0.33	+0.67	+0.5	0

RLOO: each sample is compared with the average of the other three; $A_i = (4/3)(R_i - \bar{R})$. ReMax uses the greedy answer “4” ($R = 1$) as the baseline, so only the wrong answers get pushed down. Loss: $-\frac{1}{4} \sum_i A_i \log \pi_{\theta}(y_i | x)$, with $\log \pi_{\theta}(y_i | x)$ the sum of the response's token log-probs. GRPO also divides by the group's std (0.5 here, population), giving ± 1.0 .

Common pitfalls

- No baseline with always-positive rewards**
every sampled action gets pushed up; learning is slow and noisy
- Gradient flowing through the advantage**
A must be stop-gradient; a learned b must not be trained by the policy loss
- Several steps on one batch, no ratio**
after the first step the samples are off-policy; that's what PPO fixes
- Time-limit cutoffs read as full episodes**
 G_t misses the rest of the episode; bootstrapping fixes it (actor-critic)
- LLM** Dividing each response's loss by $|y_i|$
changes the objective: long wrong answers are penalized less per token
- LLM** Groups where all k samples score the same
zero advantage, wasted compute: raise k or filter those prompts

Formulas

Generic unless tagged. sg(·) is stop-gradient. For an LLM, s_t = (x, y_{<t}) and a_t = y_t.

1 • Setup and objective

trajectory	$\tau = (s_1, a_1, r_1, \dots, s_T, a_T, r_T)$
its probability	$p_\theta(\tau) = \rho(s_1) \prod_{t=1}^T \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$
objective	$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)], \quad R(\tau) = \sum_{t=1}^T \gamma^{t-1} r_t$
LLM as a bandit	$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot \mid x)}[R(x, y)]$

2 • The policy gradient

log-derivative trick	$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$
dynamics cancel	$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
total-return form	$\nabla_\theta J = \mathbb{E} \left[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]$
reward-to-go form	$\nabla_\theta J = \mathbb{E} \left[\sum_t G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right], \quad G_t = \sum_{k=t}^T \gamma^{k-t} r_k$
exact with $\gamma < 1$	$\nabla_\theta J = \mathbb{E} \left[\sum_t \gamma^{t-1} G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]$
Most implementations drop the γ^{t-1} factor. That's standard practice, but it is no longer the exact gradient of J (Nota and Thomas, 2020). With $\gamma = 1$, as for LLMs, the issue disappears.	

3 • Baselines and variance

why it's unbiased	$\mathbb{E}_{a \sim \pi}[b(s) \nabla_\theta \log \pi_\theta(a \mid s)] = b(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$
with a baseline	$\nabla_\theta J = \mathbb{E} \left[\sum_t (G_t - b(s_t)) \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right]$
best constant b	$b^* = \frac{\mathbb{E}[\ \nabla_\theta \log p_\theta(\tau)\ ^2 R(\tau)]}{\mathbb{E}[\ \nabla_\theta \log p_\theta(\tau)\ ^2]}$
learned baseline	$L_V(\psi) = \mathbb{E}[(V_\psi(s_t) - G_t)^2] \quad (\text{Monte Carlo target})$
normalize	$\hat{A} = (A - \mu_A) / (\sigma_A + 10^{-8})$
A V_ψ fit to Monte Carlo returns keeps this REINFORCE with baseline. Using $r_t + \gamma V(s_{t+1})$ instead of G_t (bootstrapping) turns it into actor-critic.	

4 • The estimate and the loss you backprop

Monte Carlo estimate	$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_t \hat{A}_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})$
surrogate loss	$L(\theta) = -\frac{1}{N} \sum_i \sum_t \text{sg}(\hat{A}_{i,t}) \log \pi_\theta(a_{i,t} \mid s_{i,t}) - c_H \mathbb{E}_t[\mathcal{H}_t]$
check	$-\nabla_\theta L = \hat{g} + c_H \nabla_\theta \mathbb{E}_t[\mathcal{H}_t]$
On-policy: $\hat{g}$ is unbiased only for samples from the current θ . Reusing them needs $\rho = \pi_\theta / \pi_{\text{old}}$, as in PPO.	

ROBOT 5 • Gaussian policy (continuous actions)

log-density	$\log \pi_\theta(a \mid s) = -\sum_{j=1}^d \left[\frac{(a_j - \mu_j)^2}{2\sigma_j^2} + \log \sigma_j \right] - \frac{d}{2} \log 2\pi$
gradient, mean	$\frac{\partial \log \pi}{\partial \mu_j} = \frac{a_j - \mu_j}{\sigma_j^2}$
gradient, log σ	$\frac{\partial \log \pi}{\partial \log \sigma_j} = \frac{(a_j - \mu_j)^2}{\sigma_j^2} - 1$
A > 0 pulls μ toward the sampled action, A < 0 pushes it away (backprop carries this into θ). The σ term widens exploration after good unusual actions and narrows it after bad ones.	

LLM 6 • Softmax policy over tokens

sequence log-prob	$\log \pi_\theta(y \mid x) = \sum_{t=1}^{ y } \log \pi_\theta(y_t \mid x, y_{<t})$
gradient, logits	$\nabla_z \log \pi(a \mid s) = \mathbf{e}_a - \pi(\cdot \mid s)$
sequence form	$\nabla_\theta J = \mathbb{E}[(R(x, y) - b(x)) \nabla_\theta \log \pi_\theta(y \mid x)]$
KL-shaped reward	$R_\beta(x, y) = R(x, y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$
Per token, the sampled logit moves by +A(1 − π(a)) and every other logit by −A·π(a'). In R_β the log-ratio is treated as a fixed number (no gradient through it).	

LLM 7 • Baselines for k samples per prompt

RLOO	$A_i = R_i - \frac{1}{k-1} \sum_{j \neq i} R_j = \frac{k}{k-1} (R_i - \bar{R})$
ReMax	$A_i = R_i - R(x, \hat{y}_{\text{greedy}})$
GRPO, one update	$A_i = (R_i - \bar{R}) / \text{std}(R_1, \dots, R_k)$
length handling	sum over tokens, or $\div y_i $ (common, but it biases toward length)
If all k samples score the same, every group baseline gives A = 0 and the prompt teaches nothing.	

8 • Relatives

vanilla policy gradient	REINFORCE + learned V_ψ baseline + normalized advantages
actor-critic (A2C)	replace G_t with $r_t + \gamma V_\psi(s_{t+1})$ (bootstrapped)
PPO	adds $\rho = \pi_\theta / \pi_{\text{old}}$ and clipping to reuse a batch; at $\rho = 1$ it is REINFORCE
rejection-sampling FT	SFT on samples with $R = 1$: REINFORCE with $A \in \{0, 1\}$

9 • Diagnostics

returns	mean and std of G or R per batch
gradient	$\ \hat{g}\ $ and its spread across batches (variance)
exploration	entropy $\mathbb{E}[\mathcal{H}_t]$, or σ for Gaussian policies
LLM LLM extras	KL to π_{ref} , share of zero-advantage groups, response length